AI for Citizen Services and Document Processing

My technical project report

Scope: Multilingual citizen-service intent detection and OCR-assisted document processing with human review safeguards.

1. Executive summary

In this project, I built a small, runnable model of an AI-enabled citizen-services workflow. It demonstrates two complementary lanes: a text assistant that recognizes common service requests, and a document reader that extracts fields from scanned forms. Both lanes expose confidence information and route low-confidence cases to a human officer.

I designed this as a demonstration and learning project using invented data, a small training set, local processing, and synthetic sample documents. It is not connected to official records and must not be used to make real administrative decisions.

Key capabilities

- Intent classification for English, Hinglish, and Hindi text.
- Service guidance with required documents and next steps.
- OCR with per-word confidence estimates.
- Field extraction and regular-expression format validation.
- Automatic acceptance only when all required fields pass checks.
- Human handoff for uncertain, invalid, or out-of-scope requests.

2. System overview

Component	Implementation	Purpose
Citizen assistant	TF-IDF character n-grams + logistic regression	Classify requests and provide service guidance.
Service knowledge base	Local Python dictionary	Map recognized intents to documents, steps, and Hindi guidance.
Document reader	Tesseract OCR + Pillow preprocessing	Read text from scanned form images.

Component	Implementation	Purpose
Field validation	Regular expressions and threshold checks	Detect malformed or low-confidence values.
Decision layer	Human-review rule	Auto-accept clear documents; escalate uncertain cases.

I kept the implementation transparent so that each stage can be run from the command line and inspected in the Python source files.

3. Workflow design

Citizen assistant workflow

Step	Description
1. Request	A citizen enters a typed request in English, Hinglish, or Hindi.
2. Intent	The classifier predicts one of the supported service categories or out-of-scope.
3. Confidence	The prediction confidence is displayed to the operator or user.
4. Guidance	The local knowledge base returns documents and process steps.
5. Handoff	Low-confidence or unsupported requests are handed to an officer.

Document reader workflow

Step	Description
1. Scan	A form image is supplied to the command-line demo.
2. OCR	Tesseract identifies words and reports word-level confidence.
3. Extract	Expected labels such as Name and Mobile are matched to values.
4. Validate	Values are checked against expected formats.
5. Decide	All clear fields are auto-accepted; any flagged field triggers review.

4. Technical implementation

Citizen assistant

The assistant uses character-level TF-IDF features with logistic regression. Character n-grams help tolerate spelling variation and transliterated Hindi (Hinglish). The training examples cover services such as income certificates, birth certificates, ration cards, pensions, passports, driving licences, complaints, and bill or tax payments.

The classifier is deliberately small and local. The service response is not generated from an external model; it is selected from a fixed knowledge base. This makes the demo predictable and easy to audit, while also limiting its coverage.

Document reader

The reader converts an image to grayscale, applies autocontrast and mild sharpening, and passes the result to Tesseract. OCR words are grouped into lines. Each expected field is extracted from a line in the form `Label: value`.

A field is accepted only when its value matches the configured format and its average OCR confidence is at least 85 percent. Missing fields, malformed values, and lower-confidence values are marked **needs review**.

5. Demonstration and observations

Installation and execution

My implementation uses the Python packages listed in `requirements.txt` and requires the system Tesseract executable.

Command	Purpose
python3 -m venv .venv	Create an isolated Python environment.
.venv/bin/python -m pip install -r requirements.txt	Install Python dependencies.
.venv/bin/python make_samples.py	Generate clean and poor-quality sample forms.
.venv/bin/python demo.py	Run the scripted assistant and OCR demonstration.
.venv/bin/python demo.py chat	Interact with the citizen assistant.
.venv/bin/python demo.py doc samples/sample_poor.png	Process one document image.

Expected behavior

- Supported service requests return an intent, confidence score, and practical guidance.
- An unrelated weather request is outside scope and is handed to an officer.
- The clean synthetic form is intended to pass all field checks.
- The poor-quality synthetic form is intended to produce review flags.

Exact OCR confidence values can vary with the installed Tesseract version, fonts, image libraries, and operating system. The report therefore treats the results as qualitative demonstration behavior rather than fixed benchmark values.

6. Limitations and responsible-use considerations

- The intent model is trained on roughly 90 short examples, so unusual wording may be misclassified.
- Speech recognition and automatic translation are not implemented.
- The document reader expects a simple label-and-value layout rather than complex real-world forms.

- No verification against government records is implemented.
- OCR confidence is an indicator of recognition quality, not proof of truth.
- All sample data is fictional; no personal or official data should be added to this demo without appropriate controls.

A production system would require privacy controls, secure integration with authorized records, audit logging, accessibility testing, multilingual evaluation, bias monitoring, operational monitoring, and clearly defined human accountability.

7. Roadmap

Priority	Extension	Expected benefit
High	Expand and evaluate multilingual training data.	Improve classification coverage and fairness.
High	Add automated tests and reproducible evaluation metrics.	Detect regressions and measure accuracy.
High	Add secure audit logs and explicit review queues.	Support accountability and operational oversight.
Medium	Use layout-aware document models.	Handle complex government forms.
Medium	Add a Streamlit or Gradio interface.	Make the prototype easier to demonstrate.
Medium	Add speech input and translation.	Improve accessibility for more citizens.
Future	Integrate approved verification APIs.	Reduce manual verification while preserving authorization controls.

8. Conclusion

I used this project to show how a citizen-service assistant and an OCR document workflow can be combined with confidence thresholds and human oversight. Its strongest design feature is the explicit handoff rule that prevents uncertain predictions and document reads from being presented as reliable decisions.

If I developed this further, I would focus on disciplined evaluation and governance: larger representative datasets, secure data handling, measurable quality targets, and a review process that keeps human officers accountable for final outcomes.

Implementation: Python, scikit-learn, Tesseract OCR, and Pillow.